

Equipment Checkout Tracker

Code Study Guide

Every file, what it does, and why it exists. Prepared for the CRUD.IT Solutions developer exam walkthrough.

How to use this guide

The exam says you must be able to explain every line. That does not mean memorising the code. It means being able to answer three questions about any file someone points at:

1. What does this file do?
2. Why does it exist at all?
3. Why is it written this way and not the obvious alternative?

This guide is organised in four parts. Part 1 is the mental model. Part 2 walks the build order so you can explain why things were built in that sequence. Part 3 is the file-by-file reference, which is the part to revise. Part 4 is a cheat sheet of the questions most likely to come up.

If you only have thirty minutes, read Part 1 and Part 4.

PART 1 - The mental model

The three layers

The whole application is three layers stacked on top of a database.

Layer 1 is the database. A single table called items, with CHECK constraints that reject bad data no matter who writes it.

Layer 2 is the data access layer, src/lib/. Five functions plus a validation schema. This is the only place in the entire project that contains SQL.

Layer 3 is everything the user sees: pages, components, and the REST API.

The rule that holds it together: nothing outside src/lib/ contains SQL, and nothing outside items.ts imports db.ts. If a page ever imports db.ts directly, the layering has been broken.

Why the layering matters

Four concrete payoffs, not architecture for its own sake.

One. Every query lives in one file. When a query needs changing there is no hunting.

Two. The storage engine is swappable. If Neon had been unreachable on exam day, you could rewrite items.ts against an in-memory array using the same five function signatures, and nothing else in the app would change.

Three. Reads skip the network. Server Components call the functions directly instead of going out over HTTP.

Four. It is testable without a browser. listItems() is just a function.

The read path and the write path are different

This is deliberate and it is one of the most likely questions.

READS - a page load:

```
page.tsx (Server Component)
  calls listItems() directly, no HTTP
  items.ts
    db.ts
      Neon Postgres
        rows come back as snake_case
      rowToItem() converts to camelCase
    rendered as HTML on the server
```

WRITES - a form submission:

```
Browser (Client Component)
  fetch POST /api/items
  Route Handler
    validation.ts checks the body
      invalid -> 400 with field errors, stop
      valid -> continue
    items.ts
      db.ts
        Neon Postgres
          CHECK constraints get the last word
      201 Created + Location header
    router.push() + router.refresh()
```

The asymmetry is the point. Reads happen on the server, so a server calling its own HTTP API would be a pointless network round trip to reach code already in the same process. Writes come from the browser, so they

have to cross the network anyway, which is where the HTTP verbs live.

Server Components versus Client Components

The default is Server. A component only needs "use client" if it uses React state, effects, browser APIs, or DOM event handlers.

Three components in this project use it, and each has exactly one reason:

Navbar.tsx	usePathname() - a hook, needs a client runtime
ItemForm.tsx	useState, onSubmit, fetch, useRouter
DeleteButton.tsx	useRef, useState, onClick, fetch
error.tsx	Next requires error boundaries to be Client Components

Everything else - the dashboard, the table, the badges, the filter, the detail page, the stats - renders on the server and ships no JavaScript at all.

The part people get wrong: "use client" marks a boundary going DOWN the tree, not up. A Client Component does not make its parent client and does not make its siblings client. layout.tsx stays a Server Component and renders Navbar as a small client island. The pages passed in as children are unaffected.

The date decision, which shows up in six files

This is worth understanding because it connects the database to the input field.

A checkout date has no time on it. Stored as a timestamp it goes through timezone conversion, and a value near midnight renders as the previous day for someone in another timezone. So:

schema.sql	column type is DATE, not TIMESTAMPTZ
items.ts	to_char(checkout_date, 'YYYY-MM-DD') forces it back to a string
types.ts	checkoutDate is typed string, not Date
validation.ts	z.iso.date() validates a YYYY-MM-DD string
format.ts	parse and format both pinned to UTC
ItemForm.tsx	input type="date" reads and writes YYYY-MM-DD natively

The value crosses six layers as the same string with zero conversions. That is also what makes it survive being passed from a Server Component to a Client Component, because only plain data can be serialized across that boundary - a Date object could not be.

The colour palette, and why dark mode needed almost no code

Components do not use Tailwind's slate scale directly. They use six semantic tokens defined in globals.css:

--page	the app background
--surface	cards, table, nav, footer
--ink	primary text
--muted	secondary text
--line	borders and dividers
--line-strong	stronger borders

Those are redefined under a .dark class. Because @theme inline keeps them as var() references rather than resolving them at build time, switching the class re-colours the entire app at runtime.

The payoff: bg-surface, text-ink and border-line already work in both themes, so there are NO dark: variants anywhere except the badge colour maps, where each status genuinely needs its own hue in each theme.

Had the components stayed on bg-white and text-slate-900, dark mode would have meant a dark: variant on roughly every className in the project.

PART 2 - The build order, phase by phase

Understanding why things were built in this order is a separate question from what each file does, and it is a good one to be ready for.

Phase 1 - Scaffold and deploy

Ran create-next-app, pushed to GitHub, imported to Vercel, confirmed the live URL loaded the default template.

Why first: deployment is the riskiest step. A build failure on day one costs ten minutes; the same failure an hour before submission costs the exam. The exam's own tips say to deploy first.

What to remember: the `--disable-git` flag mattered, because without it create-next-app runs `git init` and writes its own commit, and the exam grades commit history.

Phase 2 - Database, schema, seed

Created the Neon project, wrote `db/schema.sql` and `db/seed.sql`, ran them in the Neon SQL Editor, wired `DATABASE_URL` into `.env.local` and Vercel.

Why second: the data has to exist before anything can read it.

What to remember: in-memory state and JSON files do not persist on Vercel. Each request can hit a different serverless instance and the filesystem is read-only outside `/tmp`. That is the whole reason for a real database.

Phase 3 - Data access layer

Built `src/lib/`: `types.ts`, `validation.ts`, `db.ts`, `items.ts`.

Why third: every later phase depends on these five functions. Build the foundation once.

How it was verified: `src/app/page.tsx` was temporarily replaced with a five-line component that dumps `listItems()` as JSON. That proved the whole chain worked before any UI existed.

Phase 4 - REST API route handlers

Built `src/app/api/items/route.ts` and `src/app/api/items/[id]/route.ts`.

Why before the UI: the endpoints could be tested with `curl` before a single pixel existed. That means any bug found while building a page afterwards is a UI bug by elimination.

This is the phase that satisfies the exam's "proper HTTP methods" requirement.

Phase 5 - Root layout and navigation

`layout.tsx`, `Navbar.tsx`, and a cleaned-up `globals.css`.

Why here: every following page needs a frame to sit in. Putting the nav in the root layout means it is impossible to forget on a page, which is what "consistent nav across all pages" asks for.

Phase 6 - Dashboard

`page.tsx`, `ItemTable`, `StatusBadge`, `ConditionBadge`, `EmptyState`, `format.ts`.

First visible feature. Replaced the throwaway JSON dump.

Phase 7 - Status filter

FilterBar, plus reading searchParams in page.tsx.

Small, self-contained, added to a page that already worked. This is the first place the URL becomes application state.

Phase 8 - Create form

ItemForm.tsx and items/new/page.tsx.

The first page that writes, and the first component that genuinely needs "use client". Built with an optional item prop specifically so Phase 10 could reuse it.

Phase 9 - Detail page

items/[id]/page.tsx and not-found.tsx.

The item-name links added in Phase 6 finally resolve.

Phase 10 - Edit page

items/[id]/edit/page.tsx. One file, about 57 lines, and none of it is form logic. That is the payoff of the Phase 8 decision.

Why after create: so the form component is reused rather than rewritten. A second copy of a 278-line form is the most common way CRUD code rots.

Phase 11 - Delete with confirmation

DeleteButton.tsx, wired into the detail page.

Destructive action last of the CRUD set.

Phase 12 - Loading, errors, responsive pass

loading.tsx, error.tsx, then a systematic check of every page at 375px and a tab through every interactive element to confirm focus rings.

Why polish once at the end rather than per page: it is one consistent pass instead of guessing at each page in isolation.

Phase 13 - README

Setup instructions, tech choices, what you would improve. A graded deliverable in its own right.

Phase 14 - Visual pass

Brand accent token, logo mark in the nav, dots on the status badges, softer cards, and the dashboard summary tiles.

This is also where a duplicate was removed: "Add Equipment" was appearing three times on an empty dashboard - nav link, page header button, and empty-state CTA. The action moved into the nav as a single persistent button.

Phase 15 - Search and sort

Search by item name or assigned person, and sort by date, condition or name.

Everything lives in the URL, so the three controls compose: searching while filtered keeps the filter, and filtering while searching keeps the search. That is what lib/url.ts exists for.

Phase 16 - Toast notifications

A toast provider in the root layout, and calls from the form and the delete button.

It has to live in the layout so its state survives router.push() - that is what lets an "item created" toast appear on the page you navigate TO.

Phase 17 - Dark mode toggle

The semantic palette described in Part 1, a toggle in the nav, and an inline script in layout.tsx that applies the saved theme before first paint.

Done last on purpose: it touches every component, so doing it earlier would have meant redoing it as each new component was added.

PART 3 - File by file

Configuration files

package.json

What it does: declares the project's dependencies and scripts.

Why it matters in the walkthrough: only two runtime dependencies were added on top of the Next.js starter - @neondatabase/serverless and zod. Everything else is the framework.

Two script details specific to Next 16:

"dev": "next dev" has no --turbo pack flag. Turbopack became the DEFAULT bundler in Next 16. On Next 15 this script would read next dev --turbo pack. Webpack is now the opt-in, via next dev --webpack.

"lint": "eslint", not "next lint". Next 16 removed the next lint command. The consequence that actually matters: next build no longer runs linting, so Vercel will not fail your build on a lint error. You have to run npm run lint yourself.

tsconfig.json

What it does: TypeScript compiler configuration.

The one line to know is the path alias:

```
"paths": { "@/*": [".src/*"] }
```

That is why imports read @/lib/items and not ../../lib/items. It also means @/lib/items resolves to src/lib/items - a file placed at src/app/lib/items would not be found, which is a real mistake that happened during the build.

.env.example and .env.local

What they do: same variable name, different values, opposite purposes.

.env.example is COMMITTED on purpose and contains a placeholder. It documents which variables the app needs, so someone cloning the repo does not get a cryptic crash.

.env.local is NEVER committed and contains the real Neon connection string. It is matched by .env* in .gitignore.

Why the connection string must contain -pooler: Vercel runs the app as serverless functions that can scale to many concurrent instances. The pooled endpoint puts PgBouncer in front of Postgres so those short-lived connections do not exhaust the database's connection limit. The direct endpoint works perfectly on localhost and then fails under load in production.

.gitignore

The line that matters is line 34: .env*

That is broader than the usual .env*.local. It ignores .env.example too, which is why that file had to be force-added with git add -f. Once a file is tracked, gitignore no longer applies to it, so later edits stage normally.

Database files

db/schema.sql

What it does: creates the items table with its constraints and indexes. Run once, by hand, in the Neon SQL Editor.

Why it is a committed file rather than something typed into a console: it is the record of what the database looks like. Anyone cloning the repo can recreate it.

Five decisions to be ready to defend:

UUID rather than SERIAL for the primary key. An incrementing integer in a URL leaks how many records exist and lets anyone walk /items/1, /items/2, /items/3. `gen_random_uuid()` is built into Postgres 13+ core, so no `pgcrypto` extension is needed.

NOT NULL is not enough for text. NOT NULL accepts " " - three spaces is a perfectly valid string. CHECK (`length(trim(item_name)) > 0`) is what actually rejects it. This was proved by running the bad insert and watching it fail.

TEXT plus CHECK rather than a native Postgres ENUM. Both reject invalid values, but a CHECK is one ALTER TABLE to change, whereas adding or reordering ENUM values is much more painful. Four statuses could plausibly become five.

DATE rather than TIMESTAMPTZ for `checkout_date`. Covered in Part 1.

`updated_at` is set by the UPDATE statement in `items.ts`, not by a database trigger. A trigger would be hidden behaviour with nothing visible in the code to point at.

Honest note on the two indexes: at five rows Postgres will ignore them and sequentially scan, because scanning a handful of rows is cheaper than consulting an index. They are there because they match the query patterns at real scale. Being able to say that out loud is worth more than the indexes themselves.

db/seed.sql

What it does: deletes every row, then inserts five sample items, then SELECTs them back as verification.

Why it is a RESET and not an append: re-running it returns the database to a known demo state, which is exactly what you want right before a walkthrough.

Why these five rows specifically. They are chosen to exercise every branch the UI has:

- all four statuses appear, so every badge colour is visible
- all four conditions appear
- one row has NULL notes, which proves the optional field renders when absent
- one row is dated 2025, which proves date sorting is not accidentally correct

Two SQL details: DATE '2026-07-28' is an explicit type cast. NULL without quotes means absent; 'NULL' with quotes would be the four-character string.

src/lib - the data layer

src/lib/types.ts

What it does: defines the vocabulary. What a Status is, what a Condition is, and what an Item looks like once it has crossed out of the database.

Why it exists separately from `validation.ts`: this is compile-time only. TypeScript types are erased when the code runs, so the only thing here that survives into the running app is the two arrays.

The line worth understanding:

```
export const STATUSES = ["Available", "Checked Out", ...] as const;
```

```
export type Status = (typeof STATUSES)[number];
```

as `const` is what makes this work. Without it TypeScript widens the array to `string[]` and `Status` degrades to plain `string`. With it, the array is a readonly tuple of literal types, so `(typeof STATUSES)[number]` - "the type of any element of this array" - becomes the union `"Available" | "Checked Out" | "Under Repair" | "Retired"`.

The payoff: one array serves as both the runtime list you `.map()` into dropdown options AND the compile-time union type. Write them separately and they drift.

Why `Item` has no `createdAt` or `updatedAt` even though both columns exist: they are there for stable ordering and future auditing. Nothing in the UI displays them, so they do not belong in the type the UI consumes.

src/lib/validation.ts

What it does: one Zod schema that runs at RUNTIME and actually inspects incoming data.

Why it is not redundant with `types.ts`. This is the most likely question about these two files, so be ready.

TypeScript types do not exist at runtime. This compiles without complaint and is a lie:

```
const item: Item = await request.json();
```

`request.json()` returns whatever bytes arrived over the network. Annotating it as `Item` does not check anything, it just tells the compiler to stop asking. Someone can curl your API with `{"status": "Banana"}` and TypeScript is perfectly happy.

Zod runs at runtime and looks at the data. So:

```
types.ts      catches YOU writing item.itemName
validation.ts catches a stranger POSTing garbage to a public API
```

They are not written twice: `ItemInput` is derived from the schema with `z.infer`, so one definition produces both.

Three details:

`z.iso.date()` checks calendar validity, not just shape. It accepts `"2026-07-28"` and rejects `"2026-02-31"`, which a regex alone would let through.

`.trim()` mutates the output, it does not just test. `" iPad "` comes out of `safeParse` as `"iPad"`, which is why the code always uses `result.data` and never the raw input.

`id` is deliberately not in the schema. A client is never allowed to choose its own primary key.

How errors are read:

```
z.flattenError(result.error).fieldErrors
-> { itemName: ["Item name is required"], status: ["Select a valid status"] }
```

That shape drops straight into a form as `errors.itemName?.[0]`, and it is the same shape the API returns on a 400 - deliberately, so client and server failures render through identical markup.

src/lib/db.ts

What it does: turns one environment variable into a sql function. That is the whole file.

Why the guard earns its four lines: without it, a missing `DATABASE_URL` produces an opaque driver error on the first query. With it you get a message naming the actual problem. This proved its worth in practice - a Vercel deploy failed at BUILD time with `"DATABASE_URL is not set"` instead of deploying an app that returned mysterious 500s on every page.

That is worth stating clearly: a failed build is the correct outcome when the app cannot possibly work. Do not make the client lazy just to get a green build.

Why the neon() HTTP driver rather than pg over a TCP socket: each query is one self-contained HTTP request, so there is no connection to open, hold, or leak on a platform that can freeze or recycle a function instance mid-execution.

src/lib/items.ts

What it does: the only file in the project containing SQL. Six exported functions plus two private helpers.

```
listItems({ status })    -> Item[]           dashboard, GET /api/items
getItem(id)              -> Item | null       detail, edit, GET /api/items/[id]
createItem(input)        -> Item             POST /api/items
updateItem(id, input)    -> Item | null       PUT /api/items/[id]
deleteItem(id)           -> boolean          DELETE /api/items/[id]
getStats()               -> counts           dashboard stat tiles
```

Six things to understand here.

ONE. `${id}` in a sql template is NOT string interpolation.

It looks identical to a template literal, but sql is a TAGGED template. The driver receives the static SQL and the values as separate arguments and sends them separately to Postgres, so `${id}` becomes `$1`. A hostile id like `'; DROP TABLE items; --` arrives as a VALUE to compare against, never as SQL. That is the entire injection defence, and it is why there is no escaping code anywhere in the file.

TWO. `to_char(checkout_date, 'YYYY-MM-DD')` is load-bearing.

Postgres drivers commonly parse DATE columns into JavaScript Date objects. A Date is a timestamp, so it lands at local midnight and can render as the previous day in another timezone - exactly the bug the DATE column was chosen to avoid. Casting it back to a string in SQL keeps the guarantee.

THREE. RETURNING saves a round trip.

INSERT ... RETURNING hands back the created row in the same query, so there is no insert-then-select. On DELETE it does double duty as the existence check.

FOUR. The return types ARE the HTTP contract.

null from `getItem` becomes a 404. false from `deleteItem` becomes a 404 instead of a 204. `createItem` returns Item and not Item | null because a create either works or throws. These are not stylistic - get them wrong and the route handlers cannot tell "missing" from "succeeded".

FIVE. The self-disabling filter.

```
WHERE (${status}::text IS NULL OR status = ${status})
```

A tagged template cannot build dynamic SQL, so instead the condition is written to be always-true when the parameter is NULL. Pass null and the first half short-circuits, making the whole clause a no-op. One query serves both the unfiltered and the filtered case. The `::text` cast is required because Postgres cannot infer a type for a bare NULL parameter.

SIX. The UUID guard.

```
if (!UUID_RE.test(id)) return null;
```

Without it, `getItem('banana')` makes Postgres raise "invalid input syntax for type uuid" - a 500 where a 404 is correct. Checking the shape first turns `/items/banana` into a clean "not found".

Two private helpers:

`rowToItem()` is the ONE place `snake_case` becomes `camelCase`. Postgres returns `item_name`, React wants `itemName`. Rename a column and this is the only function that changes.

normaliseNotes() collapses both undefined and "" to NULL. An empty notes box arrives as "" after Zod trims it, and "" is not the same thing as NULL in a nullable column.

And in getStats():

count(*) returns BIGINT in Postgres, and drivers hand bigint back as a STRING because a JS number cannot safely hold the full range. ::int casts it so it arrives as a real number - without that, "5" + "2" would concatenate to "52" instead of adding.

GROUP BY only returns statuses that have at least one row, so the code starts from zero for all four and fills in what came back. Otherwise "Retired: 0" would vanish from the UI entirely.

src/lib/format.ts

What it does: turns "2026-07-28" into "28 Jul 2026".

Why it is eight lines and not one. Two separate traps:

```
new Date(`${iso}T00:00:00Z`)    parse pinned to UTC
{ timeZone: "UTC" }            format pinned to UTC
```

Without both, new Date("2026-07-28") parses as UTC midnight and formatting it in a timezone behind UTC prints the 27th. That is the DATE column decision undone at the very last step.

The hard-coded "en-GB" locale is for a different reason: this renders on the server and hydrates on the client. If the two runtimes picked different locales they would produce different strings and React would report a hydration mismatch.

src/lib/url.ts

What it does: builds dashboard links that carry all three filters.

Why it exists: the dashboard has status pills, a search box and a sort select, and every link has to preserve the other two. Without a single builder, clicking a status pill would silently throw away your search. One function means that logic exists once.

It also omits sort=date-desc, because that is the default and leaving it out keeps the URL clean.

Note it uses URLSearchParams rather than string concatenation, which is what handles the encoding - two of the four statuses contain a space that has to become %20.

The sort query in items.ts

Worth its own note because the SQL is unusual.

You cannot parameterise an ORDER BY clause. The value is a sort DIRECTION and COLUMN, not a value to compare against, so it cannot be a \$1. The solution is a chain of CASE expressions:

```
ORDER BY
CASE WHEN $1 = 'name'      THEN item_name      END ASC,
CASE WHEN $1 = 'condition' THEN <rank>         END ASC,
CASE WHEN $1 = 'date-asc'  THEN checkout_date  END ASC,
CASE WHEN $1 = 'date-desc' THEN checkout_date  END DESC,
created_at DESC
```

Each CASE evaluates to NULL when it is not the selected sort. All-NULL means every row ties on that term, so it becomes a no-op and ordering falls through to the next line. Only one is ever active, and the parameter is still a bound value.

Sorting by condition uses an explicit rank - New 1, Good 2, Fair 3, Poor 4 - because alphabetical would give Fair, Good, New, Poor, which is not what anyone means by "sort by condition".

Search uses ILIKE, which is LIKE but case-insensitive. The % wildcards are added in JavaScript so the pattern stays a single bound parameter.

src/app/api - the REST API

src/app/api/items/route.ts

What it does: handles the COLLECTION - the things you do to "all items".

```
GET  /api/items      list, supports ?status=
POST /api/items      create one
```

Why Route Handlers and not Server Actions: the exam allows either, but it also says "proper HTTP methods". Server Actions are always an HTTP POST underneath no matter what they do, so they cannot demonstrate PUT or DELETE. Route Handlers make the verb explicit and give you an API testable with curl before any UI exists.

Details:

An unrecognised ?status= returns 400 rather than being silently ignored. Returning every item would quietly answer a different question than the one asked.

request.json() throws on malformed JSON, and that is a 400, not a 500. The server is fine; the request was not.

Validation runs here even though the form validates too, because /api/items is a public URL and anyone can curl it. Client-side validation is a convenience for honest users, never a security control. The browser is not a trust boundary.

201 Created is returned rather than 200, with a Location header pointing at the new resource. More precise, and it tells the client where the thing it just made lives.

500 responses say nothing useful. The real error goes to console.error server-side, because database errors can leak schema details.

src/app/api/items/[id]/route.ts

What it does: handles a SINGLE resource.

```
GET    /api/items/[id]  read one
PUT    /api/items/[id]  replace one
DELETE /api/items/[id]  remove one
```

The signature detail specific to Next 15+:

```
{ params }: { params: Promise<{ id: string }> }
const { id } = await params;
```

params is a Promise now and must be awaited. Forgetting gives "params should be awaited before using its properties", which is a known upgrade trap.

Why PUT and not PATCH: the edit form submits EVERY field on every save, so the request genuinely is a full replacement. PATCH would be correct for a partial change such as a status-only dropdown.

Why 204 on delete: the row is gone, so there is nothing meaningful to return. A body on a 204 is invalid HTTP, which is why that response uses new NextResponse(null, ...) instead of NextResponse.json().

Why the second DELETE of the same id returns 404: it proves the first one actually removed something, and that the handler can tell "gone" from "never existed". That distinction is the entire reason deleteItem() returns a boolean instead of void.

Two things you get free from Next:

Any verb you do not export returns 405 Method Not Allowed automatically. PATCH on this route 405s with no code from you.

OPTIONS is auto-implemented with a correct Allow header based on the verbs you did export.

And one version fact: Route Handlers are NOT cached by default in Next 16. In Next 14 GET handlers WERE cached by default and this would have needed `export const dynamic = "force-dynamic"`. Adding it now would be cargo-culting from an older version.

src/app - pages and framework files

src/app/layout.tsx

What it does: the root shell every page renders inside. Fonts, metadata, nav, content column, footer.

Why the nav lives here: putting it in the root layout means it is impossible to forget on a page, which is exactly what "consistent nav across all pages" asks for.

Details:

`LayoutProps<"">` is a Next 16 generated type. Next generates per-route prop types from your actual file structure, so children and any params are typed automatically. It does not need importing.

`flex flex-col` on body plus `flex-1` on main is the sticky-footer pattern: the footer sits at the bottom of the viewport on short pages and below the content on long ones, with no fixed positioning and no JavaScript.

`max-w-5xl mx-auto` centres the column and stops lines becoming unreadably wide on a large monitor.

`px-4 sm:px-6` is the mobile-first gutter. The unprefixed class is the phone case; prefixed classes only ever add. Writing it the other way round means fighting your own overrides.

src/app/globals.css

What it does: imports Tailwind and defines the theme. Four meaningful lines.

```
@theme {
  --color-brand: #7c3aed;
  ...
}
```

`@theme` generates utilities, so `bg-brand`, `text-brand`, `ring-brand` and `border-brand` all work. Defining the accent once here means changing it later is one line rather than a find-and-replace across thirteen files.

```
@theme inline {
  --font-sans: var(--font-geist-sans);
}
```

This maps the CSS variables `next/font` sets on the `html` element into Tailwind's theme, which is what makes the `font-sans` utility resolve to Geist. inline matters: without it Tailwind tries to resolve the value at build time, but `next/font` only sets it on the element at runtime.

Worth knowing: the scaffold shipped a `body { font-family: Arial }` rule that overrode the Geist font entirely - the font was downloaded on every page load and never used. Removing it and putting `font-sans` on body was a real bug fix found by reading the generated code rather than trusting it.

Also removed: the scaffold's `@media (prefers-color-scheme: dark)` block. It only inverted the page background while leaving cards and text light, which looks broken rather than themed. Dark mode is a bonus feature and belongs behind a proper toggle.

src/app/page.tsx - the dashboard

What it does: the required dashboard. Lists all equipment with status badges, filters by status, shows an empty state when there is no data.

It is an async Server Component. It awaits the database directly - no `useEffect`, no loading state, no fetch to its own API. The HTML arrives with the data already in it.

Details:

```
const searchParams = await props.searchParams;
```

`searchParams` is a Promise in Next 15+, exactly like `params`. `PageProps<"/">` is the generated helper; the actual definition is `searchParams: Promise<Record<string, string | string[] | undefined>>`.

`parseStatus()` treats the query string as hostile. `?status=anything` arrives as an ordinary string typed by whoever holds the URL bar, so it is checked against `STATUSES` and anything unrecognised falls back to no filter. It also handles the array case, because `?status=A&status=B` arrives as an array, not a string.

```
const [items, stats] = await Promise.all([listItems({ status }), getStats()]);
```

`Promise.all` so the two queries run concurrently. They do not depend on each other, so awaiting them in sequence would just add the second round trip to the page load.

The stats are deliberately `UNFILTERED`. They summarise everything, so they stay stable while you filter. If they changed with the filter, "Available: 2" would just repeat the row count.

src/app/loading.tsx

What it does: the skeleton shown while the dashboard's database query runs.

Why there is no `isLoading` flag anywhere in your code: Next wraps the segment in a `Suspense` boundary automatically and uses this file as the fallback. Creating the file is the entire wiring.

Why a skeleton and not a spinner: it reserves roughly the right space so the page does not jump when real content replaces it. It also makes Neon's cold start look deliberate instead of broken.

Note on keys: index keys are fine here even though they are avoided in the item list. This array is fixed-length and never reorders, so there is no identity for React to lose.

src/app/error.tsx

What it does: the error boundary for the segment.

It **MUST** be a Client Component. That is a hard Next requirement, not a preference - the boundary attaches on the client.

The detail worth knowing: in `PRODUCTION`, Next strips the real error message before it reaches the browser and sends only a digest, which is a hash matching the full error in the server logs. That is deliberate, because error messages leak schema and file paths. So rendering `error.message` to the user is pointless. Log it, show something generic, and show the digest so it can be quoted.

`reset()` re-renders the segment. It is a genuine retry of the failed render, not a page reload.

Scope limit: `error.tsx` does NOT catch errors thrown by `layout.tsx`, because the layout renders outside the boundary. That needs `global-error.tsx`, which replaces the whole document. Not added here.

src/app/not-found.tsx

What it does: rendered whenever `notFound()` is called anywhere, and for any URL matching no route.

Why it lives at the app root: there it sits inside layout.tsx, so the 404 still has the nav and footer and the user is not stranded on a bare error page. Placed at src/app/items/[id]/not-found.tsx it would only cover that one route.

src/app/items/new/page.tsx

What it does: the create page. A Server Component with nothing async in it.

Why it is still a Server Component when it renders a client form: it has no data to fetch, it just renders ItemForm inside a heading and a card. Keeping it server means the page shell ships no JavaScript; only ItemForm crosses the boundary.

src/app/items/[id]/page.tsx - the detail page

What it does: shows every field for one item, with Edit and Delete.

Three things to understand.

ONE. notFound() sends a REAL 404.

It throws, Next catches it, renders not-found.tsx, and sets the HTTP status to 404. The alternative - returning `<p>Not found</p>` from the component - looks identical in the browser but sends 200 OK, which tells crawlers and monitoring the page exists. Status codes are part of the response, not decoration.

It also does work for TypeScript: because it never returns, item narrows to non-null on every line after it, with no `!` and no cast. Calling it as a bare statement rather than returning its result is what makes that narrowing happen.

TWO. `cache()` from React collapses two queries into one.

`generateMetadata` and the page component both need the same record. Next dedupes `fetch()` automatically, but `getItem` is a plain async function calling a SQL driver, so Next has no way to know two calls are equivalent. `cache()` makes it explicit, memoised for the lifetime of one request. Without it every detail page load runs the same query twice.

THREE. The page calls `getItem()` directly, not `fetch("/api/items/...")`.

This is the read path. The page already runs on the server, so going out over HTTP to reach a function in the same process is a round trip for nothing.

Also: `whitespace-pre-wrap` on the notes, because HTML collapses every run of whitespace - newlines included - into a single space. Without it, multi-line notes render as one paragraph.

src/app/items/[id]/edit/page.tsx

What it does: loads the item and hands it to ItemForm. About 57 lines, and none of it is form logic.

The entire edit feature is one line:

```
<ItemForm item={item} />
```

Because ItemForm was built with an optional item prop, passing one in switches the same component from POST to PUT, pre-fills every field, relabels the button, and redirects to the detail page instead of the dashboard.

The concept to be ready for: the **SERIALIZATION BOUNDARY**.

`EditItemPage` is a Server Component and `ItemForm` is a Client Component, so item gets serialized to cross between them - the server embeds it in the payload and the browser reconstructs it. Only plain data survives: objects, strings, numbers, null, arrays. NOT functions, NOT class instances, and NOT Date objects.

That is the payoff of typing `checkoutDate` as a string. Had it been a Date, this prop would have been a problem to work around.

The pattern to name: the Server Component fetches, the Client Component receives via props. The client never fetches its own initial data. No `useEffect`, no spinner, no flash of an empty form.

src/components

src/components/Navbar.tsx - CLIENT

What it does: the logo mark, the wordmark, and the Add Equipment button.

Why it is a Client Component: `usePathname()` is a hook, and hooks need a client runtime. That is the ONLY reason. It is used to hide the Add Equipment button when you are already on `/items/new`, where it would be redundant.

Why the action lives in the nav rather than the dashboard header: with only two real pages, a nav whose sole link is "Dashboard" is pointless - the logo already does that. Putting the action in the nav also means you can add an item from the detail page, which otherwise has no route to it. It also removed a duplicate: "Add Equipment" was appearing three times on an empty dashboard.

src/components/StatusBadge.tsx and ConditionBadge.tsx - SERVER

What they do: map a value to a coloured pill.

The single most important thing in these files:

```
const STYLES: Record<Status, string> = {
  "Available": "bg-green-100 text-green-800 ring-green-600/20",
  ...
};
```

Tailwind CANNOT see class names built at runtime. It scans your source files for COMPLETE LITERAL STRINGS at build time. A template like `bg-${colour}-100` generates no CSS whatsoever, because that string never appears in the source for Tailwind to find. The class silently does not exist and the badge renders unstyled. Nothing errors. This is the most common Tailwind mistake.

Second point: typing it as `Record<Status, string>` makes the map EXHAUSTIVE. Add a fifth status to `types.ts` and this object stops compiling until that status gets a colour. The type system enforces that the UI keeps up with the data model.

The condition palette is deliberately lighter than the status palette. Status is the primary signal on the dashboard; giving both the same visual weight would leave the user with nothing to look at first.

The dot has `aria-hidden="true"` because it repeats information the text already carries. A screen reader announcing "bullet Available" is noise.

src/components/ItemTable.tsx - SERVER

What it does: renders the item list twice, in two different markup shapes, and lets CSS pick.

```
hidden md:block      a real <table>, desktop only
md:hidden            stacked cards, mobile only
```

Why two shapes rather than one that stretches: a five-column table genuinely cannot work at 375px. Because the switch is pure CSS it costs no JavaScript and no resize listener.

`key={item.id}` and NEVER the array index. With an index key, deleting a row makes React reuse the wrong DOM node for the row that shifts up into its place.

`scope="col"` on the table headers tells a screen reader those cells label columns, so it can announce "Status: Available" per cell.

<dl> for the mobile cards because they are genuinely label/value pairs, not a list of items. The markup describes the data, not just the layout.

src/components/FilterBar.tsx - SERVER

What it does: four status pills plus "All", each a Link to a different query string.

This one is worth explaining carefully, because the plan originally called for a Client Component using useRouter and useSearchParams, and it was deliberately built the other way.

A filter is just navigation, and navigation is links. Consequences:

"use client"	not needed
JavaScript shipped	none
Works with JS off	yes
Back button	works
Shareable URL	works, by construction

A <select> would earn its place if the filter had to combine with a live search box. For four fixed values, links are strictly simpler.

Two details: "All" is the ABSENCE of the parameter, not ?status=All - it keeps the unfiltered dashboard on a clean "/" and means there is no magic string to special-case. And encodeURIComponent is not optional, because two of the four statuses contain a space which must become %20 or the value never matches.

src/components/EmptyState.tsx - SERVER

What it does: covers TWO different empty conditions.

"No results for this filter" and "nothing exists yet" look identical but mean opposite things. Offering "Add Equipment" to someone who has five items and just picked a filter matching none of them is misleading - what they want is "Clear filter".

Why an empty state is worth a component at all: a blank page reads as BROKEN. "No equipment yet" plus a link to the create form reads as working-but-empty, and points at the next action.

src/components/StatsCards.tsx - SERVER

What it does: five display tiles - total plus one per status.

They are DISPLAY ONLY, not links. They were briefly built as links to ?status=, which duplicated the FilterBar sitting directly below them - two controls doing the same job. The tiles summarise; the pills filter.

src/components/ItemForm.tsx - CLIENT

The largest component in the project, and the only one used by two different pages.

What it does: create and edit, in one component. The presence of the item prop decides which.

	create	edit
item prop	absent	present
initial values	empty / defaults	from item
request	POST /api/items	PUT /api/items/{id}
button	"Create item"	"Save changes"
redirect	/	/items/{id}

Why not two components: every future change to a field, a validation rule or a style would have to be made in two places, and eventually would not be. That is the most common way CRUD code rots.

Why this component genuinely needs "use client" - four independent reasons: useState for the values, onSubmit for the handler, fetch for the request, useRouter for the redirect.

The HTTP reason, which is the good answer: an HTML `<form>` can only send GET or POST. PUT is not available to it. Using the correct verb for an edit therefore requires fetch, and calling fetch from an event handler requires a client runtime. The HTTP requirement and the "use client" requirement are the same decision, not two.

Six details:

`event.preventDefault()` stops the browser doing its own full-page form submission, which would mean the fetch never runs.

`noValidate` on the form turns off the browser's native validation bubbles. Without it you get browser popups for some fields and inline messages for others - two different UIs for the same problem.

The same schema runs client-side and server-side, and BOTH produce `{ field: [messages] }`. That is deliberate, so client and server failures land in the same state and render through identical markup. One error path, not two.

The submit button disables on submit and NEVER re-enables on success. POST is not idempotent - two clicks create two items. It stays disabled through the navigation because re-enabling during the transition reopens exactly that window. On failure it does re-enable, because the user needs to retry.

`router.push()` then `router.refresh()`. The push navigates; the refresh re-runs the Server Components so the dashboard renders with the new row. `revalidatePath` in the route handler marked the cache stale, but something still has to ask for a fresh render.

`values.notes ?? ""` on the textarea, because notes is optional so it can be undefined, and a controlled input that receives undefined makes React switch it to uncontrolled and warn.

And one thing that is absent on purpose: the checkout date does not default to today. Computing "today" in component state would run during server render AND again on the client, and in different timezones those produce different dates - a hydration mismatch.

Accessibility, which is easy to skip and was not: every input has a `<label htmlFor>`, errors are wired to their field with `aria-invalid` and `aria-describedby`, and the form-level error banner uses `role="alert"` so it is announced when it appears. A red border communicates nothing to a screen reader on its own.

src/components/DeleteButton.tsx - CLIENT

What it does: a Delete button that opens a confirmation dialog before doing anything.

Why the NATIVE `<dialog>` element rather than `window.confirm()` or a hand-rolled modal:

	<code>confirm()</code>	hand-rolled	<code><dialog></code>
Styleable	no	yes	yes
Focus trapped	n/a	you build it	BROWSER
Escape closes	yes	you build it	BROWSER
Above everything	yes	z-index war	top layer
Blocks main thread	YES	no	no

`showModal()`, not `show()`, is what makes it modal - focus trapping, Escape, top-layer rendering and the `::backdrop` pseudo-element all come from the browser. Getting focus management subtly wrong by hand is the norm, not the exception.

Why Delete is a `<button>` and NEVER an `<a href>`: DELETE is not a safe method. Browsers, link prefetchers and crawlers follow links speculatively, so a destructive action behind an anchor eventually fires with nobody having clicked it. This is the concrete consequence of the safe/unsafe distinction.

Why the 204 response must not be parsed as JSON: 204 No Content is success with no body by definition. `await response.json()` on an empty response throws - and the delete has already succeeded at that point, so the user would see an error for an action that worked.

Why 404 gets its own message: it is a real race, not defensive padding. Another tab or another person can delete the item between this page rendering and the button being clicked. Test it with two browser tabs.

Why `router.push("/")` comes BEFORE `router.refresh()`: the current page is about to stop existing, so leave it first, then re-render the dashboard without the row.

Why `useRef` and not `useState` for the dialog: its open/closed state is owned by the DOM element and exposed through the imperative `showModal()` and `close()` methods. Mirroring that in React state would create two sources of truth that can disagree.

src/components/SearchSortBar.tsx - SERVER

What it does: the search box and sort dropdown, in one form.

The interesting part: it is a SERVER component despite being a form. It uses `next/form`, which extends `<form>` with client-side navigation on submit while remaining a real form underneath. So it needs no `useState`, no `onChange`, no client bundle - and it still works with JavaScript disabled, because the browser just does an ordinary GET navigation.

Two details worth knowing:

`defaultValue`, not `value`. These are uncontrolled inputs. The URL is the state, not React. Using `value` without an `onChange` would make them read-only.

The active status rides along as a hidden input. The status pills are outside this form, so without that hidden field, searching would clear the filter.

src/components/ToastProvider.tsx - CLIENT

What it does: a context provider plus the fixed viewport that renders the toasts.

Why it lives in the root layout: its state has to survive `router.push()`. `ItemForm` fires a toast and then immediately navigates - if the provider were inside the page, it would unmount with the page and the toast would never be seen. In the layout, it persists across client-side navigation.

The thing to be ready for: wrapping `{children}` in a Client Component does NOT make those children client components. They are passed in as a prop from a Server Component, already rendered. The provider only wraps them.

`aria-live="polite"` on the viewport announces new toasts to a screen reader without stealing focus.

src/components/ThemeToggle.tsx - CLIENT

What it does: flips the `.dark` class on `<html>` and remembers the choice.

It holds NO React state, and that is deliberate. An earlier version used `useState` plus an effect to read the current theme after hydration, and ESLint flagged it - `react-hooks/set-state-in-effect`. Calling `setState` synchronously in an effect causes a cascading render.

The fix was to remove the state entirely:

- the class on `<html>` is the single source of truth
- the click handler reads it from the DOM and toggles it
- the sun and moon icons are swapped by the dark: variant in CSS, so the icon can never

disagree with the actual theme

That also removes the hydration mismatch, because there is no client-only value being rendered.

The no-flash script in layout.tsx

The other half of dark mode.

```
if (localStorage.theme === 'dark' || (unset && OS prefers dark))  
  document.documentElement.classList.add('dark')
```

It is an inline `<script>` in `<head>`, so it runs BEFORE first paint. Without it the page would render light, then snap to dark once React hydrated - the "flash of wrong theme".

`<html>` carries `suppressHydrationWarning` because this script edits `className` before React sees the element, which React would otherwise report as a mismatch.

PART 4 - Cheat sheet

The questions most likely to come up, with short answers.

Architecture

Q: Why Neon and not in-memory or a JSON file? A: Neither persists on Vercel. Each request can hit a different serverless instance and the filesystem is read-only outside /tmp. In-memory works perfectly in next dev and then loses data in production, which is the worst possible failure mode.

Q: Why Route Handlers and not Server Actions? A: The exam asks for proper HTTP methods. Server Actions are always a POST underneath, so they cannot demonstrate PUT or DELETE.

Q: Why no ORM? A: The bar is explaining every line. Raw parameterised SQL needs no framework vocabulary to defend, and the tagged-template driver parameterises values anyway, so there is no injection-safety cost.

Q: Why is all the SQL in one file? A: One place to change a query, and the storage layer becomes swappable. If Neon were unreachable I could replace items.ts with an array and nothing else would change.

Q: Why do pages call items.ts directly instead of your own API? A: The page already runs on the server. Going over HTTP to reach a function in the same process is a round trip for nothing. The API exists for the browser.

HTTP

Q: What does "safe" mean? A: No server state changes. Only GET. That is why Delete is a button, not a link - prefetchers follow links speculatively.

Q: What does "idempotent" mean? A: Running it five times leaves the same result as once. GET, PUT and DELETE are; POST is not. Two POSTs create two items, which is why the submit button disables.

Q: Why PUT and not PATCH? A: The edit form submits every field, so it is a full replacement. PATCH would suit a status-only toggle.

Q: Why 204 on delete? A: Nothing meaningful to return. A body on a 204 is invalid HTTP.

Q: Why does the second DELETE return 404? A: It proves the first one removed something. That is why deleteItem returns a boolean rather than void.

Q: What happens if I PATCH your item endpoint? A: 405 Method Not Allowed, automatically. Next returns it for any verb the route file does not export, along with a correct Allow header.

Next.js

Q: When do you use "use client"? A: When the component needs React state, effects, browser APIs or event handlers. Three components here use it, each for one specific reason. Everything else is a Server Component.

Q: Does a Client Component make its parent client? A: No. The directive marks a boundary going down the tree, not up. layout.tsx stays a Server Component and renders Navbar as a client island.

Q: Why is params awaited? A: In Next 15+ params and searchParams are Promises.

Q: Why does notFound() matter over rendering a message? A: It sets a real 404 status. A rendered message sends 200 OK, which tells crawlers the page exists.

Q: What does cache() do? A: Memoises a function for one request. generateMetadata and the page both need the item; without it that is two identical queries.

Database and data

Q: Why DATE and not TIMESTAMPTZ? A: A checkout date has no time. As a timestamp it goes through timezone conversion and a date near midnight can render as the previous day elsewhere.

Q: Why isn't NOT NULL enough? A: It accepts three spaces as a valid string. The CHECK with trim and length rejects it.

Q: Why TEXT + CHECK instead of ENUM? A: A CHECK is one ALTER TABLE to change. Reordering or removing ENUM values is much harder.

Q: How are you safe from SQL injection? A: The sql tagged template sends the query and the values separately. `${id}` becomes `$1`. Nothing is ever concatenated into a query string.

Q: Why validate twice? A: The API is a public URL. Anyone can curl it. Client-side validation is a convenience, not a security control. There are CHECK constraints as a third layer.

Bonus features

Q: How does search work? A: ILIKE on `item_name` and `assigned_to`, so it is case-insensitive and matches either. The `%` wildcards are added in JavaScript so the pattern stays one bound parameter.

Q: Why can't you parameterise the sort? A: A bound parameter is a VALUE. A sort is a column and a direction, which are part of the query structure. So each option is a CASE that returns NULL when it is not selected - all-NULL ties, so that term is a no-op and ordering falls through.

Q: Why does sorting by condition need a CASE? A: Alphabetical gives Fair, Good, New, Poor. The ranks 1-4 give New, Good, Fair, Poor, which is what "sort by condition" actually means.

Q: Why is the search bar a Server Component? A: It uses `next/form`, which adds client-side navigation to a real `<form>`. No state, no event handlers, no client bundle - and it still works with JavaScript off.

Q: Why is the toast provider in the layout and not the page? A: Its state has to survive `router.push()`. In the page it would unmount before the toast was ever seen.

Q: Does wrapping children in a client provider make them client components? A: No. They are passed in as a prop from a Server Component, already rendered.

Q: How does dark mode avoid a flash on load? A: An inline script in `<head>` applies the saved theme before first paint. Doing it in React would mean rendering light first, then switching after hydration.

Q: Why does ThemeToggle have no state? A: The `.dark` class on `<html>` is the source of truth. The click handler reads and toggles it, and the sun/moon icons swap via the `dark: CSS` variant. ESLint flagged the earlier state-in-effect version, and removing the state also removed the hydration mismatch.

Q: Why are there almost no `dark:` classes? A: The palette is six semantic tokens redefined under `.dark`. `bg-surface` and `text-ink` already work in both themes. Only the badge colour maps need `dark:` variants, because each status genuinely needs a different hue per theme.

Tailwind and UI

Q: Why is there no `tailwind.config.js`? A: Tailwind v4 moved configuration into CSS. `globals.css` opens with `@import "tailwindcss"` and the build wiring is the `@tailwindcss/postcss` plugin.

Q: Why are badge colours in a lookup object? A: Tailwind scans source for complete literal class names at build time. A constructed name like `bg-${colour}-100` generates no CSS and fails silently.

Q: How does the responsive layout work? A: Two markup shapes, table and cards, switched by hidden `md:block` and `md:hidden`. Pure CSS, no JavaScript.

Final checklist before the walkthrough

Run these and know the answers:

<code>npx tsc --noEmit</code>	no type errors
<code>npm run lint</code>	no warnings
<code>npm run build</code>	succeeds
<code>npm run start</code>	click through the production build

In the Neon SQL Editor, re-run `db/seed.sql` so the demo data is in a known state.

Be able to demonstrate, live:

```
curl -i http://localhost:3000/api/items
curl -i -X DELETE http://localhost:3000/api/items/<id>      204
curl -i -X DELETE http://localhost:3000/api/items/<id>      404
curl -i -X PATCH  http://localhost:3000/api/items/<id>      405
```

That last sequence is the single best demonstration in the project. It shows the delete worked, that the handler distinguishes gone from never-existed, and that unexported verbs are handled for free.